

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI

Semester: 5

Academic Year: 2026-27

Division: C

Batch: 3

Practical No: 1

Roll no: 59

PRN No: 12413745

Name of Student: Netra Lalwani

Subject Name: Deep Learning

Problem Statement:

Install and configure TensorFlow/Keras in Google Colab. Perform data preprocessing, normalization, train-test split, and visualization on a sample dataset.

Algorithm:

Data Preprocessing, Normalization, Train-Test Splitting, and Visualization using MNIST Dataset

1. Start the program.
2. Import the required libraries (TensorFlow/Keras, NumPy, Matplotlib, Scikit-learn, KaggleHub, etc.).
3. Download the MNIST dataset using `kagglehub.dataset_download()`.
4. Load the MNIST images and labels into memory.
5. Verify the dataset dimensions and inspect the data.
6. Separate the image data (features) and digit labels (targets).
7. Normalize the pixel values by dividing each pixel by 255 so that all values lie in the range 0 to 1.
8. Split the dataset into training and testing sets using `train_test_split()`.
9. Visualize sample handwritten digit images using Matplotlib.
10. Plot the distribution of digit classes to understand the dataset.

11. Display the shapes of the training and testing datasets.

12. End the program.

Code:

```
!pip install -q kagglehub mlxtend
```

```
import kagglehub
import os
import numpy as np
import matplotlib.pyplot as plt

from mlxtend.data import loadlocal_mnist
from sklearn.model_selection import train_test_split
```

```
# Download latest version
path = kagglehub.dataset_download("hojjatk/mnist-dataset")

print("Path to dataset files:", path)
```

Using Colab cache for faster access to the 'mnist-dataset' dataset.
Path to dataset files: /kaggle/input/mnist-dataset

```
print("Files in dataset:")

for file in os.listdir(path):
    print(file)

X, y = loadlocal_mnist(
    images_path=os.path.join(path, "train-images.idx3-ubyte"),
    labels_path=os.path.join(path, "train-labels.idx1-ubyte")
)

print("Images Shape:", X.shape)
print("Labels Shape:", y.shape)

X = X.astype("float32") / 255.0

print("Minimum Pixel Value:", X.min())
print("Maximum Pixel Value:", X.max())
```

```

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Training Data:", X_train.shape)
print("Testing Data :", X_test.shape)

plt.figure(figsize=(4,4))
plt.imshow(X_train[0].reshape(28,28), cmap="gray")
plt.title(f"Digit: {y_train[0]}")
plt.axis("off")
plt.show()

plt.figure(figsize=(12,5))

for i in range(10):
    plt.subplot(2,5,i+1)
    plt.imshow(X_train[i].reshape(28,28), cmap="gray")
    plt.title(y_train[i])
    plt.axis("off")

plt.tight_layout()
plt.show()

plt.figure(figsize=(8,5))

plt.hist(y_train, bins=np.arange(11)-0.5, edgecolor="black")

plt.xticks(range(10))
plt.xlabel("Digit")
plt.ylabel("Frequency")
plt.title("MNIST Class Distribution")

plt.show()

print("Training Images :", X_train.shape)
print("Testing Images :", X_test.shape)

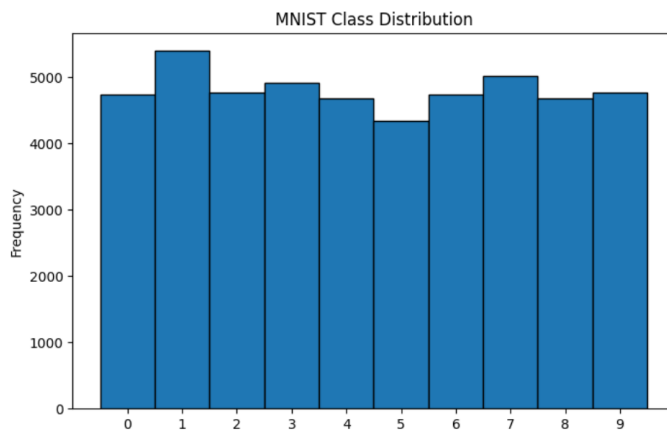
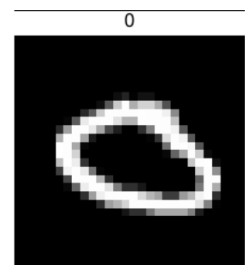
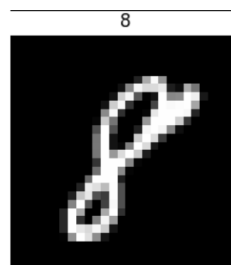
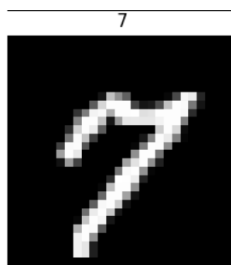
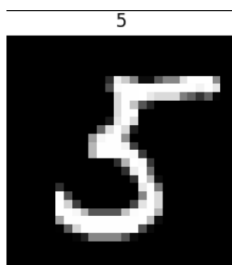
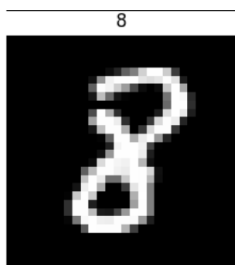
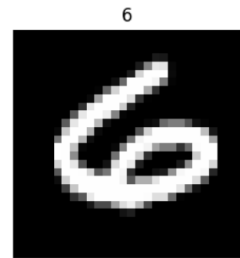
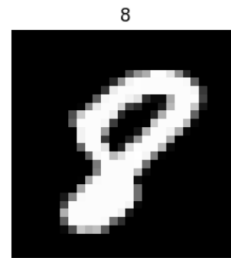
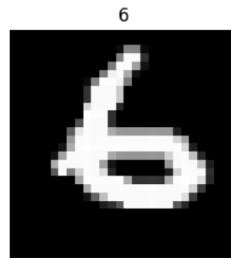
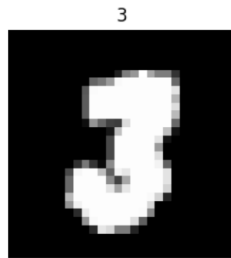
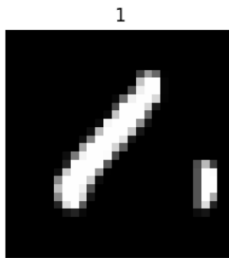
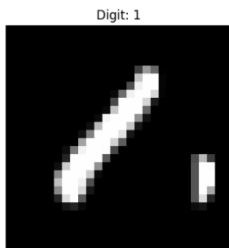
print("Training Labels :", y_train.shape)
print("Testing Labels :", y_test.shape)

print("Pixel Value Range :", X_train.min(), "to", X_train.max())

```

Output :

```
Files in dataset:      Images Shape: (60000, 784)  Minimum Pixel Value: 0.0
t10k-labels-idx1-ubyte  Labels Shape: (60000,)      Maximum Pixel Value: 1.0
train-images.idx3-ubyte
t10k-images-idx3-ubyte
t10k-labels-idx1-ubyte
t10k-images.idx3-ubyte
train-labels.idx1-ubyte
train-labels.idx1-ubyte
train-images-idx3-ubyte
```



```
Training Images : (48000, 784)
Testing Images  : (12000, 784)
Training Labels : (48000,)
Testing Labels  : (12000,)
Pixel Value Range : 0.0 to 1.0
```